

Lab 01: Installation, Setup & Docker

Operating Systems and Distributed Systems

This lab aims to get the students started with the course by installing and setting up the programs and environments we will use. Furthermore, a few introductory exercises in Docker will be provided to familiarise the students with the tool.

The lab is structured so the student is assisted through the tasks.

Remember to read the exercise and the accompanying text thoroughly before asking for help.

Contents

Exercise 01: Prepare the Environment	2
1. Unix Shell	2
Docker & Docker-Compose	3
Exercise 02: Hello, World!	4
Exercise 03: Home-cooked Hello, World!	5
Creating a Dockerfile	5
Building a Docker Image	6
Exercise 04: Building Dockerfiles	7
Dockerfile	7
Running a Docker Container	7
Exercise 05: Expose Container Contents	9
Running a Docker Container	9
Exercise 06: Make Your Script	10

Exercise 01: Prepare the Environment

Several tools will be used throughout this course to demonstrate and simulate concepts. This exercise walks the student through the installation and setup process to ensure that the following exercises can be completed.

1. Unix Shell

Several exercises make use of functionalities provided by the Operating System. Different OS' implement these functionalities in different manners. To ensure that everything is compatible with these exercises, you must have access to a Unix Shell. Many different shells, namely **sh**, **bash**, and **zsh**, often come pre-packaged with an OS.

You are set if you run either macOS or any Linux distribution – no need for more. However, if you are running Windows, you must choose how to get a functioning Unix Shell. All the following options end in installing a Linux distribution in some form, but all with different capabilities. We recommend the following order for the best experience and ease of installation.

2. Windows Subsystem for Linux (WSL)

WSL is a system that allows running Linux architecture on a Windows machine. If you are interested, you can read more here (<https://docs.microsoft.com/en-us/windows/wsl/about>).

This solution only provides command-line access to Linux, meaning you do not have any Graphical User Interface (GUI) to interact with programs. This is completely viable for this course, as we do not need the GUI capability of Linux.

For installation, follow this guide: <https://docs.microsoft.com/en-us/windows/wsl/install>.

3. Dual Boot

Dual Booting is a solution where two (or more) OS' are installed alongside each other. This solution allows you to choose which OS to start when your computer boots up.

This solution is not as straightforward as the others, as it can involve configurations specific to each computer vendor. The upside is that it gives a native experience, where you run Linux directly on your hardware without any intermediate.

This solution gives you a full-blown Linux environment with a usable OS like Windows.

WARNING: This method involves superuser changes, which, in the worst case, can result in you needing to reinstall (and potentially lose all your files).

For installation, the following guide can be used: <https://itsfoss.com/install-ubuntu-1404-dual-boot-mode-windows-8-81-uefi/>.

Virtual Machine (VM)

Lastly, it is possible to run Linux from within a VM. This method is not recommended as it is resource-heavy as you run Linux through Windows.

Firstly, a Hypervisor must be installed. We recommend the open-source Hypervisor VirtualBox: <https://www.geeksforgeeks.org/how-to-install-virtualbox-on-windows/>.

Secondly, Linux must be installed: <https://www.geeksforgeeks.org/how-to-install-ubuntu-on-virtualbox/>.

Docker & Docker-Compose

Docker is a tool for creating, building, running, and maintaining containers.

Docker can be installed either with a GUI, through Docker Desktop: <https://docs.docker.com/desktop/> or only with the command-line interface (CLI), through the Docker Engine <https://docs.docker.com/engine/install/ubuntu/>.

By default, Docker commands can only be executed by root users. To enable non-root users to execute Docker commands, follow the *Manage Docker as a non-root user* steps of the post-installation procedures: <https://docs.docker.com/engine/install/linux-postinstall/>.

Exercise 02: Hello, World!

This exercise introduces the student to Docker and how to run a container.

In Docker, a container is spun up by using the run command, `$ docker container run <DOCKER-IMAGE>`. When using the command, it is required for you to specify the image you would like to spin up. If the image is not found in your library, Docker will look for the image on Docker Hub.

On Docker Hub, an image named `hello-world` has been created. The image is used to test if Docker has been installed correctly and to give the user a feeling of how the docker CLI works.

Your task

1. Create a container that runs the above-mentioned `hello-world` image

The exercise will return the following message:

```
1 $ docker container run <DOCKER-IMAGE>
2
3 Hello from Docker!
4 This message shows that your installation appears to be working correctly.
5
6 To generate this message, Docker took the following steps:
7   1. The Docker client contacted the Docker daemon.
8   2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
9   3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
10  4. The Docker daemon streamed that output to the Docker client, which sent it to
   your terminal.
11
12 To try something more ambitious, you can run an Ubuntu container with:
13   $ docker run -it ubuntu bash
14
15 Share images, automate workflows, and more with a free Docker ID:
16   https://hub.docker.com/
17
18 For more examples and ideas, visit:
19   https://docs.docker.com/get-started/
```

Exercise 03: Home-cooked Hello, World!

In this exercise, the student will create a Dockerfile, specifying configurations for a homemade Hello World image.

In Docker, each image is an immutable file created from a Dockerfile. It is a template used to create containers with a predefined environment. Docker also features containers, a lightweight and portable encapsulation of the environment described in the Docker image.

As a metaphor, images can be seen as classes, whereas containers can be seen as objects.

IMPORTANT: A Docker container has a running (currently executing) process that defines its lifetime.

Creating a Dockerfile

A Dockerfile is simply a file called "Dockerfile" without any extension. Inside a Dockerfile, a specific syntax is used to create and describe the environment spun up with a container. For starters, there are two things to notice inside a Dockerfile. Firstly, that `#` used at the beginning of a line is treated as a comment. This means the Docker engine skips over the lines beginning with a `#`.

Secondly, the instructions in a Dockerfile follow the syntax `<INSTRUCTION> <ARGUMENTS>`. That means the instructions are written in capital letters and the arguments in non-capital letters.

```
1 # A Dockerfile often begins with the FROM instruction, telling the Docker engine
   which image this image is built from.
2 # In this case, ubuntu is used as the argument.
3 FROM ubuntu
4
5 # The RUN instruction is an instruction that takes a BASH command as an argument.
   In this way, simple functionality is included in the Dockerfile.
6 # In this case, the text "Hello, World"! is inserted into a simple "helloWorld"-
   file.
7 RUN echo "Some text" > helloWorld
8
9 # The CMD instruction is the instruction that determines the container's lifetime.
   This instruction is not run when building the image. When a container is spun
   up from an image, the following command is run as the executing command. When
   the command finishes its execution, the container will shut down.
10 # In this case, the text inside the "helloWorld"-file is printed on the screen.
11 CMD ["cat", "helloWorld"]
```

If you do not have a specific idea for the image you want to create, develop your Dockerfile from the `ubuntu` image (using `FROM ubuntu`).

Building a Docker Image

Building a docker image can be done by using the build command,

```
$ docker image build <PATH-TO-DOCKERFILE> .
```

An important option to know when building Docker images is the tag option (`-t`), which is used to tag the Docker image with a specific name. This makes it easier to run the container, as the user chooses the name and not the Docker engine.

An example of using the tag option: `$ docker image build -t <TAG> <PATH-TO-DOCKERFILE>`

Furthermore, it is important to note that by default, the Docker Engine expects the Dockerfile to be named "Dockerfile". To build a Dockerfile with another name, include the file option, `-f`.

```
$ docker image build -f <FILENAME> <PATH-TO-DOCKERFILE>
```

Your task

1. Create a Dockerfile that prints the following message to the screen: "Hello, World! 42."
2. Build a Docker Image from the Dockerfile. Remember to give the build a tag
3. Spin up a container running the above-built image

The exercise will return the following message:

```
1 Hello, World! 42.
```

Note: `PATH-TO-DOCKERFILE` is the path to the folder containing the Dockerfile, not the file itself.

Exercise 04: Building Dockerfiles

In this exercise, the student will use the included Dockerfile to build an image and spin up a container.

This Dockerfile differs from the previous files as a new instruction is introduced.

Dockerfile

In this Dockerfile a new instruction is introduced. The `COPY` instruction is used to copy files or folders from the host machine to the Docker Container.

```
1 FROM ubuntu
2
3 # Here, the COPY instruction copies the content of the /L1E3 folder from the host
  machine to the / (root) destination of the container.
4 COPY L1E3/ /home/
5
6 CMD bash
```

Running a Docker Container

Spinning up and running a Docker container follows the same way described in the previous exercise.

This time, though, an extended version of the run command is used:

```
$ docker container run <OPTIONS> <DOCKER-IMAGE> <ARGUMENT>
```

Here, many options can be used to customise the container-creation process. One of these options is the `-it` option, making it possible to run processes interactively.

Combining the `-it` option with the `bash` argument allows us to enter and interact with the container under runtime.

Your task

1. Build the included Dockerfile. Remember to give the build a tag
2. Run the above-built image. Remember to run it in a way that makes it possible to enter the container
3. Install `python3` and run the included `print.py` file

4. Edit the Dockerfile, so `python3` is installed beforehand and runs the Python script automatically

The exercise will return the following message:

1

200

Exercise 05: Expose Container Contents

In this exercise, the student will copy the included `.php-files` to an image, expose a port, and see the resulting website on the host machine. The files can be found in the resources.

Running a Docker Container

Spinning up and running a Docker container follows the same way described in the previous exercise. Another important option is for exposing ports, `-p <HOST-PORT>:<CONTAINER-PORT>`.

This option exposes port `<CONTAINER-PORT>` from the container and connects it with port `<HOST-PORT>` on the host machine. This means that a visit to the host machine's (localhost) port `<HOST-PORT>` will be redirected to the container's port `<CONTAINER-PORT>`.

```
docker container run -p <HOST-PORT>:<CONTAINER-PORT> <DOCKER-IMAGE>
```

There are many cases where this function comes in handy. An example would be if you wanted to host an Apache web server in a container. It would be boring if it were not possible for outside machines to access the information that the server serves.

In this case, we use an Apache server that communicates through HTTP. This is done through port 80.

Your task

1. Examine the included Dockerfile. Place your files so they comply with the structure dictated by the Dockerfile
2. Build the included Dockerfile. Remember to give the build a tag
3. Run the above-built image. Remember to expose the port
4. Visit the forwarded port through a browser on your (host) machine

The exercise will return the following message:

```
1 [Thu Jul 04 13:29:48.343170 2019] [mpm_prefork:notice] [pid 1] AH00163: Apache
   /2.4.25 (Debian) PHP/7.2.19 configured -- resuming normal operations
2 [Thu Jul 04 13:29:48.343489 2019] [core:notice] [pid 1] AH00094: Command line: '
   apache2 -D FOREGROUND'
3
4 172.17.0.1 - - [04/Jul/2019:13:20:20 +0000] "GET / HTTP/1.1" 200 370 "-" "Mozilla
   /5.0 (Macintosh; Intel Mac OS X 10_13_6) AppleWebKit/605.1.15 (KHTML, like
   Gecko) Version/12.1.1 Safari/605.1.15"
```

Exercise 06: Make Your Script

In this exercise, the student will create a Dockerfile and a Python script copied into the container. Python must be run inside the container as the executed process.

Help: Look at the previous Dockerfiles for inspiration on configuring the Dockerfile.

When the executed process is set to be a process with a definite lifetime, the container is stopped when the process dies. In this case, the container will stop when the script is executed.

Help: When running Python in a Docker container, use the flag `-u` to print while the container runs.

Your task

1. Create a Python script that makes some form of output and prints to the terminal Inspiration: Make a for-loop that counts. Use the `time.sleep(int count)` method to make the executing thread sleep.
2. Create a Dockerfile that copies the script to the container, installs Python, and runs the script

The exercise will return the following message:

1 SCRIPT-OUTPUT